

CUTTING PROBLEM



Découpe de barres métalliques

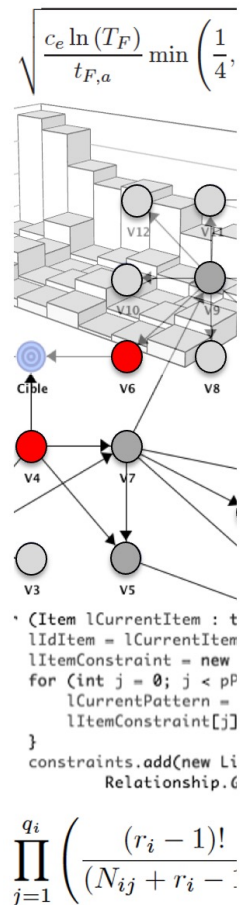
Chaque jour, une entreprise fabrique des bureaux de 3 hauteurs différentes : 40cm, 60cm et 70cm. Pour cela, elle fabrique des pieds (4 par bureau) à partir de barres métalliques de 1,50m ou de 2m. Le prix des barres est proportionnel à leurs longueurs.

Aujourd'hui, cette entreprise reçoit une commande de 108 petits bureaux, 125 moyens et 100 grands.

1. Modéliser ce problème afin de pouvoir répondre aux commandes journalières tout en minimisant les chutes (et donc le coût) ?

La modélisation doit répondre aux questions suivantes : combien de barres de chaque longueur l'entreprise doit-elle commander ? Comment, chacune des barres commandées, doit-elle être découpée ?

2. Donner la solution pour la commande du jour.



CUTING PROBLEM

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```
• (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.6
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - \dots} \right)$$

plans de découpe

1m50

2m00

#plan	Pieds de 40cm	Pieds de 60cm	Pieds de 70cm	Chute (cm)
x ₁	0	0	2	10
x ₂	0	1	1	20
x ₃	2	0	1	0
x ₄	0	2	0	30
x ₅	2	1	0	10
x ₆	3	0	0	30
x ₇	0	1	2	0
x ₈	0	2	1	10
x ₉	1	0	2	20
x ₁₀	3	0	1	10
x ₁₁	0	3	0	20
x ₁₂	5	0	0	0
x ₁₃	1	1	1	30
x ₁₄	2	2	0	0
x ₁₅	3	1	0	20

CUTING PROBLEM

Variables :

x_i : le nombre de barres découpées suivant le plan de découpe i ($i=1,\dots,15$)

Contraintes :

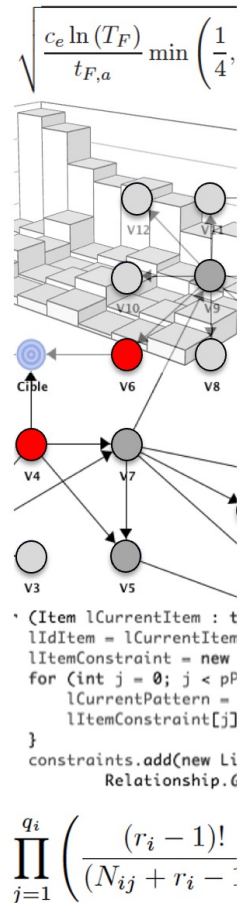
$$2x_3 + 2x_5 + 3x_6 + x_9 + 3x_{10} + 5x_{12} + x_{13} + 2x_{14} + 3x_{15} \geq 432$$

$$x_2 + 2x_4 + x_5 + x_7 + 2x_8 + 3x_{11} + x_{13} + 2x_{14} + x_{15} \geq 500$$

$$2x_1 + x_2 + x_3 + 2x_7 + x_8 + 2x_9 + x_{10} + x_{13} \geq 400$$

Fonction objectif :

$$\text{Minimiser : } \sum_{i=1}^6 150x_i + \sum_{i=7}^{15} 200x_i - 4 \times (108 \times 40 + 125 \times 60 + 100 \times 70)$$



CUTING PROBLEM

```
from ortools.sat.python import cp_model

model = cp_model.CpModel()

# variables
x = []
for i in range(15):
    x.append(model.NewIntVar(0, 1000000, f"x_{i}"))

# Contraintes
model.Add(2*x[2]+2*x[4]+3*x[5]+x[8]+3*x[9]+5*x[11]+x[12]+2*x[13]+3*x[14]>=432)
model.Add(x[1]+2*x[3]+x[4]+x[6]+2*x[7]+3*x[10]+x[12]+2*x[13]+x[14]>=500)
model.Add(2*x[0]+x[1]+x[2]+2*x[6]+x[7]+2*x[8]+x[9]+x[12]>=400)

# Fonction objectif
c = [150, 150, 150, 150, 150, 150, 200, 200, 200, 200, 200, 200, 200, 200, 200]
model.Minimize(sum((c[i]*x[i]) for i in range(15)))

# Solution
solver = cp_model.CpSolver()
status = solver.Solve(model)
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:
    for i in range(15):
        print(f"x_{i+1} = " + str(solver.Value(x[i])))
    print("z = " + str(solver.ObjectiveValue()))
else:
    print('No solution')
```

Plusieurs solutions optimales

x ₁ = 0	x ₁ = 0
x ₂ = 1	x ₂ = 0
x ₃ = 1	x ₃ = 53
x ₄ = 0	x ₄ = 0
x ₅ = 0	x ₅ = 1
x ₆ = 0	x ₆ = 0
x ₇ = 199	x ₇ = 173
x ₈ = 0	x ₈ = 1
x ₉ = 0	x ₉ = 0
x ₁₀ = 0	x ₁₀ = 0
x ₁₁ = 0	x ₁₁ = 0
x ₁₂ = 26	x ₁₂ = 0
x ₁₃ = 0	x ₁₃ = 0
x ₁₄ = 150	x ₁₄ = 162
x ₁₅ = 0	x ₁₅ = 0
z = 75300.0	z = 75300.0

x₁ = 0
 x₂ = 1
 x₃ = 53
 x₄ = 0
 x₅ = 0
 x₆ = 0
 x₇ = 173
 x₈ = 0
 x₉ = 0
 x₁₀ = 0
 x₁₁ = 0
 x₁₂ = 0
 x₁₃ = 0
 x₁₄ = 163
 x₁₅ = 0
 z = 75300.0 (20cm de chute)



Diagram illustrating a cutting problem. A 3D stack of blocks is shown with a target area (blue circle) and a cutting plane (red line). The diagram includes labels for variables (V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12) and constraints (C1, C2, C3, C4, C5, C6, C7, C8, C9, C10, C11, C12). The diagram also shows a network of nodes and edges representing the constraints.

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

(Item lCurrentItem : t
 lIdItem = lCurrentItem
 lItemConstraint = new
 for (int j = 0; j < pP
 lCurrentPattern =
 lItemConstraint[j]
 }
 constraints.add(new Li
 Relationship.C

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

2D CUTTING STOCK PROBLEM WITH SETUP COST

Optimisation d'un processus d'impression

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```
graph TD
    Cible((Cible))
    V1((V1))
    V2((V2))
    V3((V3))
    V4((V4))
    V5((V5))
    V6((V6))
    V7((V7))
    V8((V8))
    V9((V9))
    V10((V10))
    V11((V11))
    V12((V12))
    Cible --> V1
    Cible --> V2
    Cible --> V3
    Cible --> V4
    Cible --> V5
    Cible --> V6
    Cible --> V7
    Cible --> V8
    Cible --> V9
    Cible --> V10
    Cible --> V11
    Cible --> V12
```

```
*(Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - \dots} \right)$$

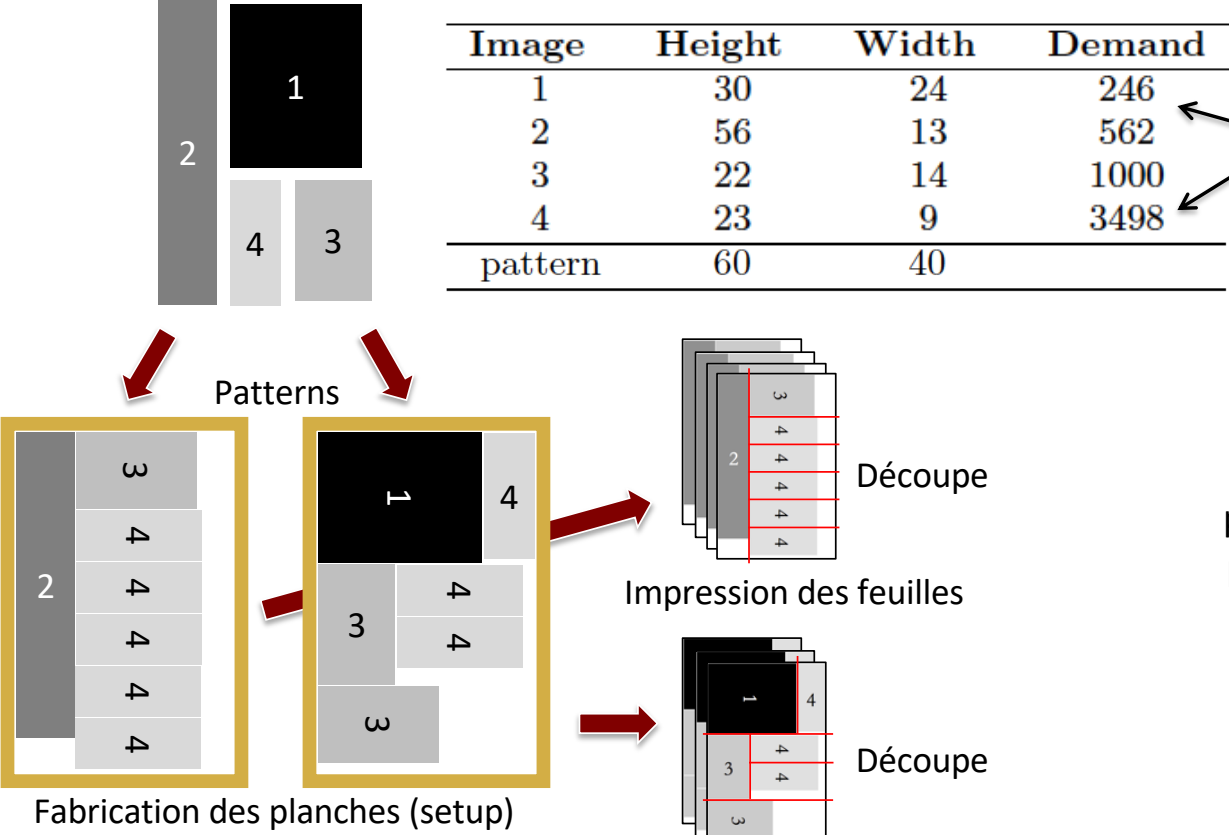


Image	Height	Width	Demand
1	30	24	246
2	56	13	562
3	22	14	1000
4	23	9	3498
pattern	60	40	

Nombre d'impressions

Objectif: satisfaire la demande du client tout en minimisant le coût total de production (fabrication des planches et impression des feuilles)

setup cost = 20\$
stock sheet cost = 1\$

2D CUTTING STOCK PROBLEM WITH SETUP COST

Une solution triviale :
Un pattern par image

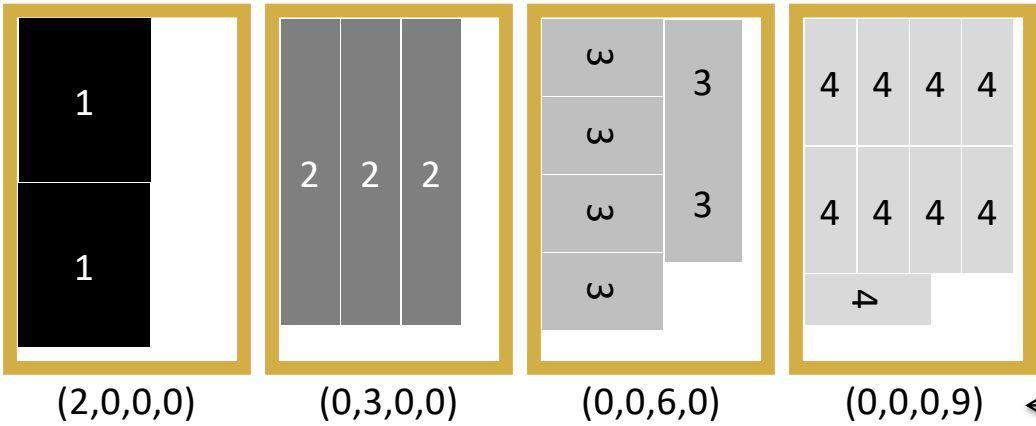
Image	Height	Width	Demand
1	30	24	246
2	56	13	562
3	22	14	1000
4	23	9	3498
pattern	60	40	

setup cost = 20\$ - stock sheet cost = 1\$

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```
* (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$



123

188

167

389

← # d'images

← # de feuilles à imprimer pour satisfaire la demande

- surproduction:
- 0 image n°1
 - 2 images n°2
 - 2 images n°3
 - 3 images n°4

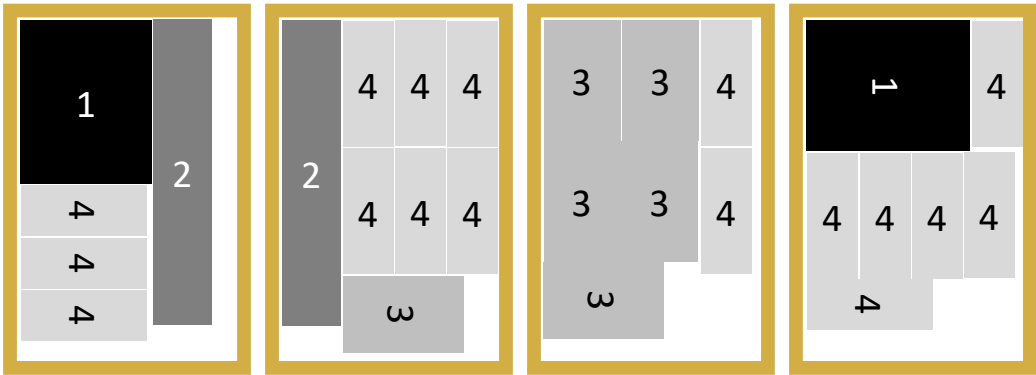
Coût total = 4×20\$ + (123+188+167+389)×1\$ = 947\$

2D CUTTING STOCK PROBLEM WITH SETUP COST

Une solution plus complexe
... obtenue avec 4 patterns

Image	Height	Width	Demand
1	30	24	246
2	56	13	562
3	22	14	1000
4	23	9	3498
pattern	60	40	

setup cost = 20\$ - stock sheet cost = 1\$



(1,1,0,3)

(0,1,1,6)

(0,0,5,2)

(1,0,0,6)

← # d'images

178

385

123

68

← # de feuilles à imprimer pour satisfaire la demande

surproduction:
- 0 image n°1
- 1 images n°2
- 0 images n°3
- 0 images n°4

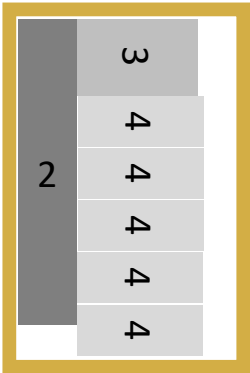
Coût total = 4×20\$ + 754×1\$ = 834\$ (au lieu de 947\$)

2D CUTTING STOCK PROBLEM WITH SETUP COST

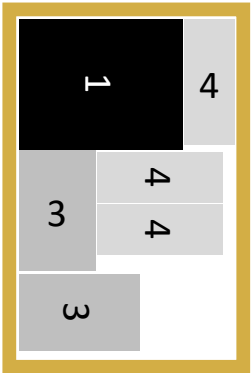
Meilleures solutions trouvées avec 2 et 3 patterns :

```
• (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
```

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$
$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

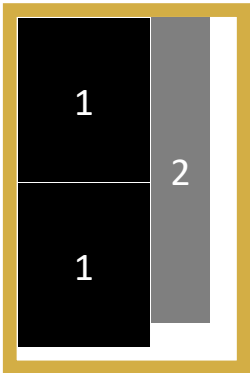


(0,1,1,5)
562

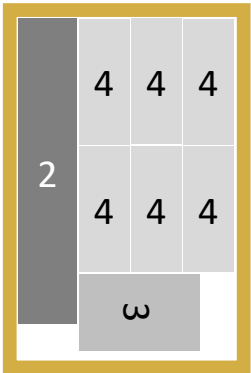


(1,0,2,3)
246

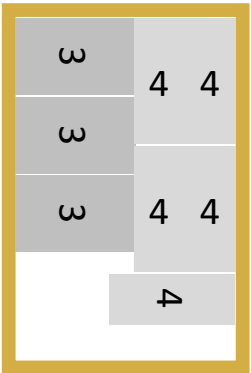
Coût = 2×20\$ + 808×1\$
= 848\$



(2,1,0,0)
123



(0,1,1,6)
439

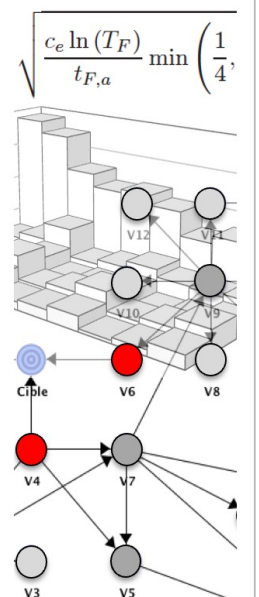


(0,0,3,5)
187

Coût = 3×20\$ + 749×1\$
= 809\$

2D CUTTING STOCK PROBLEM WITH SETUP COST

Le problème consiste à minimiser le coût total de production (fabrication des planches + impression des feuilles) tout en déterminant automatiquement ...



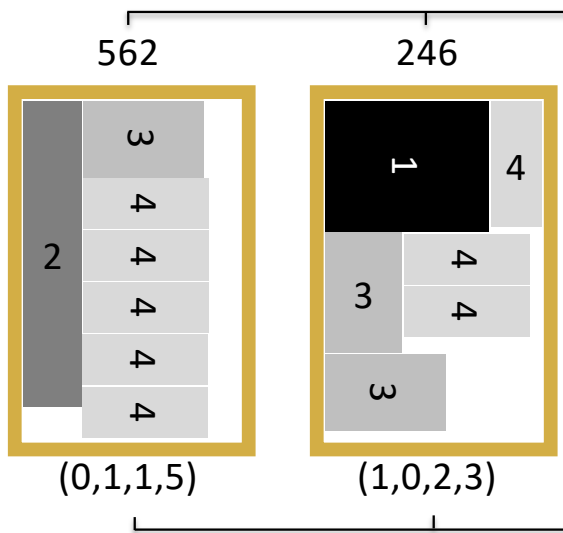
```
* (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
```

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$
$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - \dots)} \right)$$

la **position de chaque image** sur chaque pattern
(2D Bin Packing)



Contrainte de « Guillotine »



le **nombre de feuilles** à imprimer pour chaque pattern (LP)

le nombre de patterns

le **nombre de chaque image** sur chaque pattern



de trouver le nombre de chaque image sur chaque pattern (et trouver le nombre de patterns)
↳ problème combinatoire (# images > à 40)

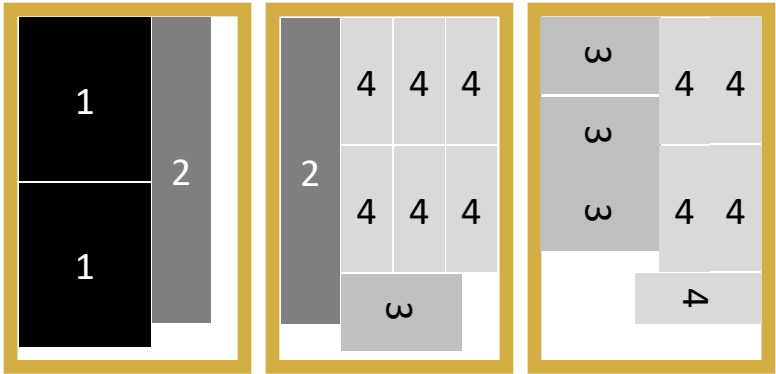
2D CUTTING STOCK PROBLEM WITH SETUP COST

Résolution par Métaheuristiques (algorithmes génétiques, recuit simulé, ...)

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```
• (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - \dots} \right)$$



(2,1,0,0) (0,1,1,6) (0,0,3,5)
x₁= 123 x₂= 439 x₃= 187

Image	Demand
1	246
2	562
3	1000
4	3498

Calcul du nombre
d'impressions de
chaque pattern par
programmation
linéaire

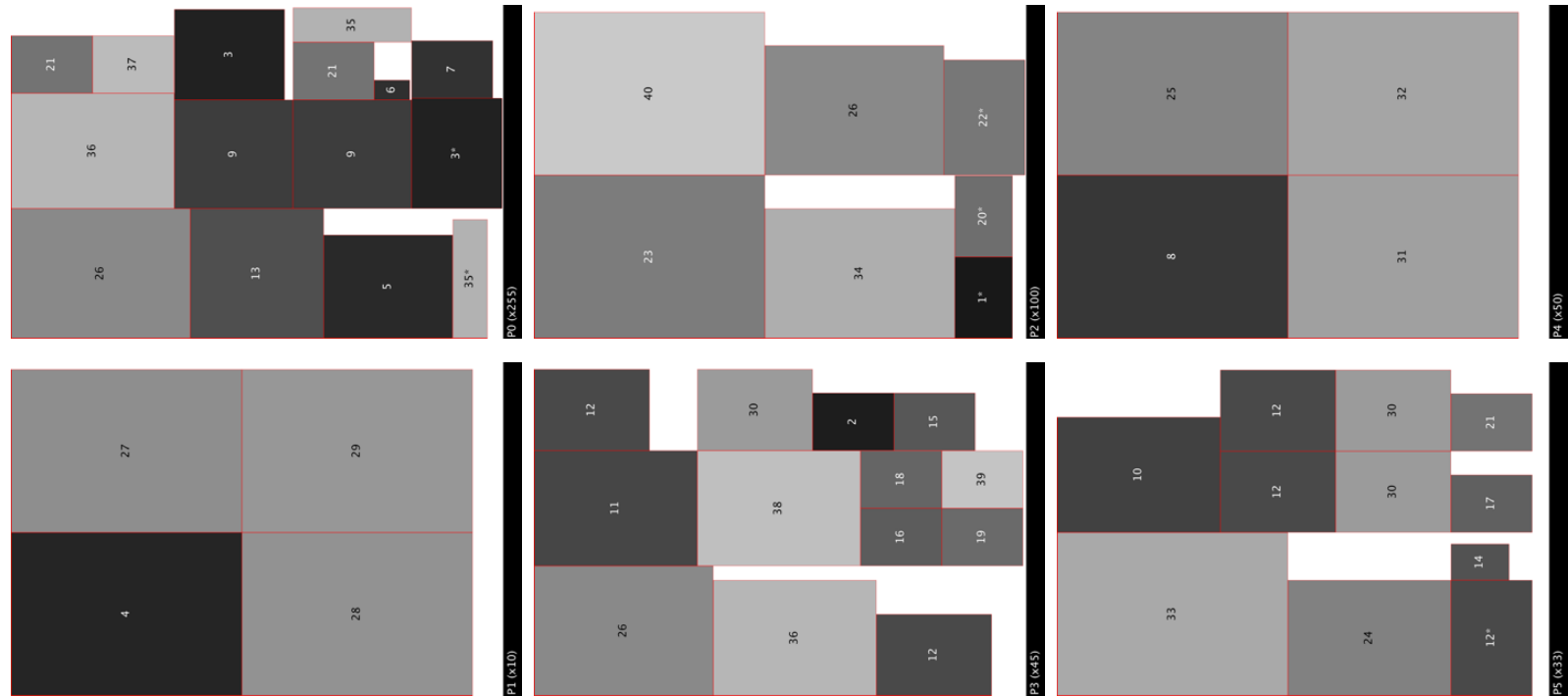
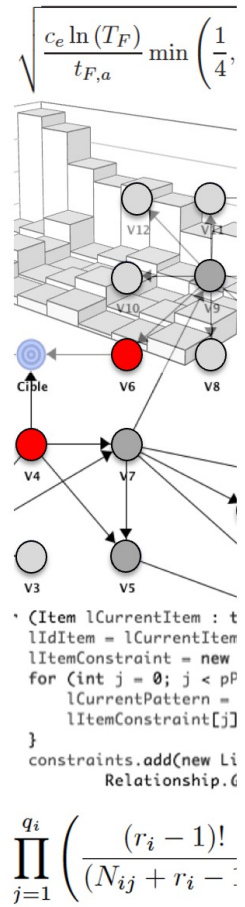
Opérateurs de voisinage :

- Ajouter une image à un pattern
- Supprimer une image dans un pattern
- Passer une image d'un pattern à un autre
- Echanger deux images de deux patterns

$$2x_1 \geq 246$$
$$x_1 + x_2 \geq 562$$
$$x_1 + 3x_3 \geq 1000$$
$$6x_2 + 5x_3 \geq 3498$$
$$\text{Min } z = x_1 + x_2 + x_3$$

2D CUTTING STOCK PROBLEM WITH SETUP COST

Exemple d'un résultat avec données réelles (40 images) :



2D CUTTING STOCK PROBLEM WITH SETUP COST

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```
* (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.6
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - \dots} \right)$$



A Genetic Algorithm to Solve a Real 2-D Cutting Stock Problem with Setup Cost in the Paper Industry

Stéphane Bonnevay
ERIC, Université de Lyon
5 av. Pierre Mendès France

Philippe Aubertin
AXOPEN
149 bd Stalingrad
Villeurbanne, France

Gérald Gavin
ERIC, Université de Lyon
5 av. Pierre Mendès France
69676 Bron Cedex, France

MIC 2015: The XI Metaheuristics International Conference

id-1

A Simulated Annealing algorithm for real-world 2-D Cutting Stock Problem with Setup Cost

Stéphane Bonnevay¹, Philippe Aubertin², Thibault Lazert¹

¹ ERIC, Université de Lyon, 5 av. Pierre Mendès France, 69676 Bron Cedex, France
stephane.bonnevay@univ-lyon1.fr

² AXOPEN, 149 bd Stalingrad, 69100 Villeurbanne, France
philippe.aubertin@axopen.com

Abstract

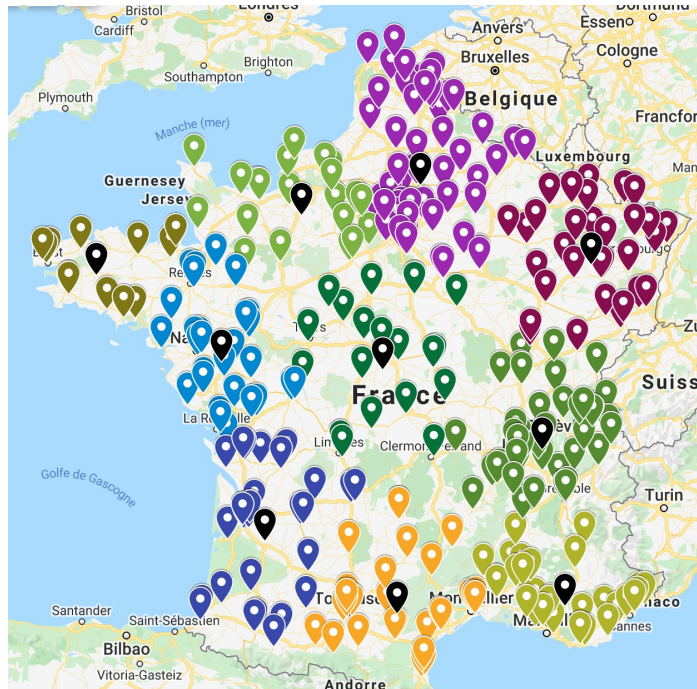
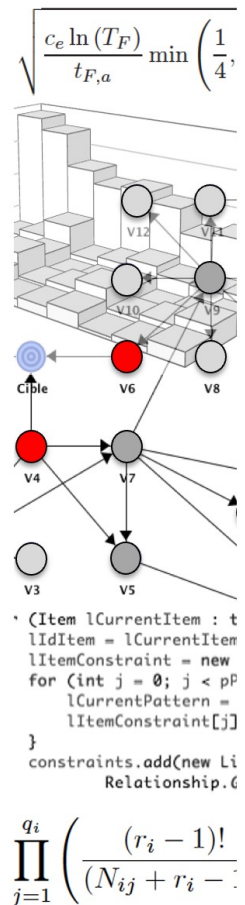
bonnevay@univ-lyon1.fr

Stock com-Pack-linear isfying the demand [1, 14]. The Cutting Stock Problem with Setup Cost (CSP-S) takes into account the cost of the setup and the cost of the stock sheet printed in order to minimize the total production cost [10, 15, 18]. Most of existing papers deal with one dimensional problems [8, 3, 6]. In this paper, we give a solution to the Two-Dimensional

SET PARTITIONING PROBLEM

Optimisation de sessions de formation

Etant donnés plus de 300 agences et n lieux de formations répartis sur le territoire, trouver une affectation des agents à ces n lieux de formations qui minimise la distance « route » totale des trajets de tous les agents.



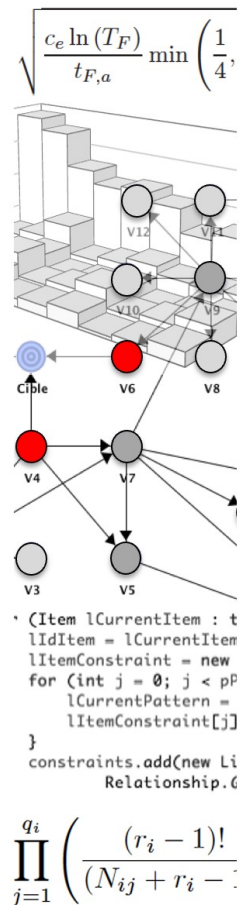
Données et contraintes :

- Coordonnées GPS des agences et des lieux de formation
- Nombre d'agents formés au sein de chaque agence
- Nombre maximum d'agents autorisés par formation
- Les agents d'une agence doivent être affectés au même lieu

FACILITY LOCATION PROBLEM

Optimisation de sessions de formation

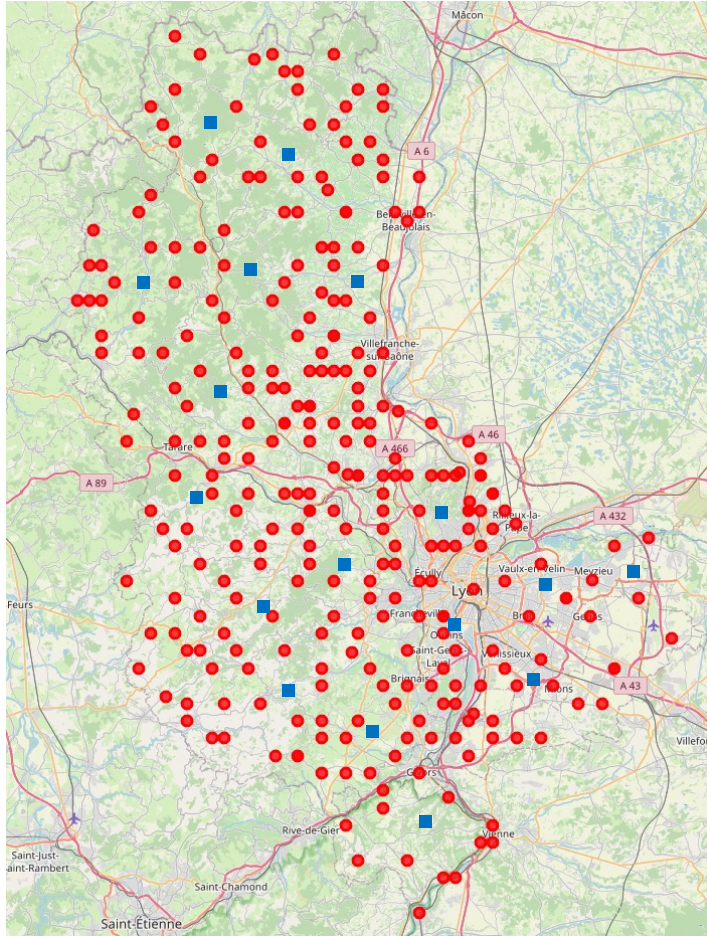
Etant donnés plus de 300 agences et n formateurs, trouver n lieux de formations et une affectation des agents à ces n lieux de formation en minimisant la distance « route » totale des trajets de tous les agents.



Données et contraintes :

- Coordonnées GPS des agences
- Nombre d'agents formés au sein de chaque agence
- Nombre maximum d'agents autorisés par formation
- Les agents d'une agence doivent être affectés au même lieu
- Lieux à trouver sont quelconques ou choisis parmi une liste de villes

FACILITY LOCATION PROBLEM



Facility location problem

Objectif : trouver les meilleurs lieux pour implanter des centres logistiques (facilities : hôpitaux, entrepôts, data centers, usines, ...) pour livrer des clients.

Soit C un ensemble de n clients ●

Soit F un ensemble de m sites ■ où potentiellement ouvrir un centre logistique

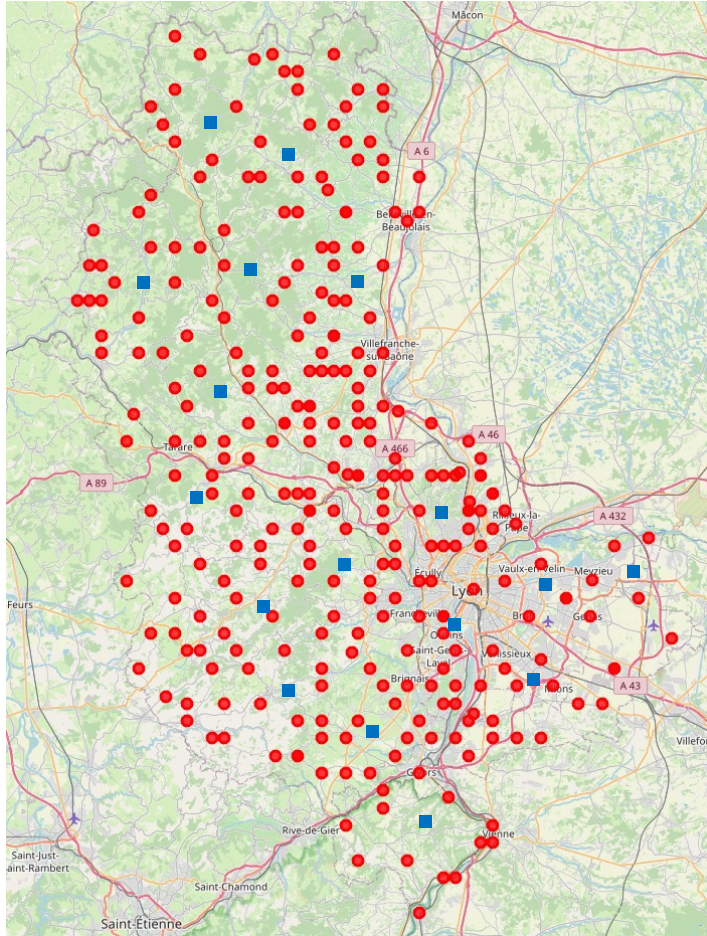
Ouvrir un site j coûte f_j

Servir un client i depuis le site j coûte c_{ij}

Chaque client doit être servi par un seul site ouvert

- ### 1. Modéliser le problème sous forme d'un PLNE

FACILITY LOCATION PROBLEM



Facility location problem

Objectif : trouver les meilleurs lieux pour implanter des centres logistiques (facilities : hôpitaux, entrepôts, data centers, usines, ...) pour livrer des clients.

Soit C un ensemble de n clients ●

Soit F un ensemble de m sites ■ où potentiellement ouvrir un centre logistique

Ouvrir un site j coûte f_j

Servir un client i depuis le site j coûte c_{ij}

Chaque client doit être servi par un seul site ouvert

Capacitated Facility location problem

Soit d_i la demande du client i

Un site j ouvert peut servir un nombre limité de clients (capacité max : q_j)

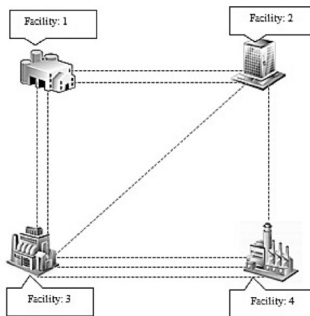
$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```

    * (Item lCurrentItem : t
      lIdItem = lCurrentItem
      lItemConstraint = new
      for (int j = 0; j < pP
        lCurrentPattern =
        lItemConstraint[j]
      }
      constraints.add(new Li
        Relationship.C
  
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

QUADRATIC ASSIGNMENT PROBLEM

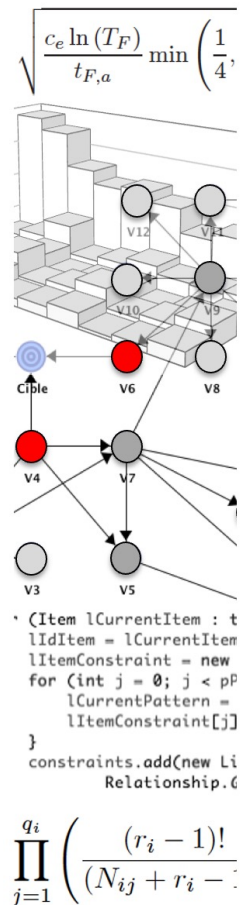


De manière générale, le problème d'affectation quadratique consiste à positionner n équipements sur n emplacements.

Soient P l'ensemble des équipements et L l'ensemble des emplacements, tous deux de taille n . On note d la distance entre les emplacements et w une fonction de poids définie entre les équipements.

L'objectif est de trouver une bijection f de P dans L , qui affecte les équipements aux emplacements, telle que :

$$\sum_{a,b \in P} w(a,b) d(f(a), f(b)) \text{ soit minimum.}$$



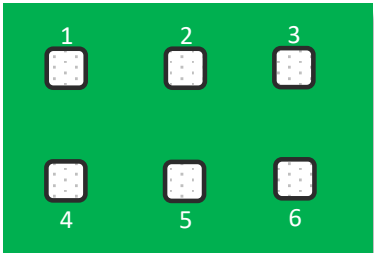
QAPLIB - A Quadratic Assignment Problem Library

R.E. BURKARD, E. ÇELA, S.E. KARISCH and F. RENDL

Welcome to the QAPLIB Home Page, the online version of
[QAPLIB - A Quadratic Assignment Problem Library](http://anjos.mgi.polymtl.ca/qaplib/)
 by R.E. Burkard, S.E. Karisch and F. Rendl,

<http://anjos.mgi.polymtl.ca/qaplib/>

QUADRATIC ASSIGNMENT PROBLEM



On veut positionner 6 composants électroniques, notés a b c d e f, sur les 6 emplacements de la carte électronique ci-contre.

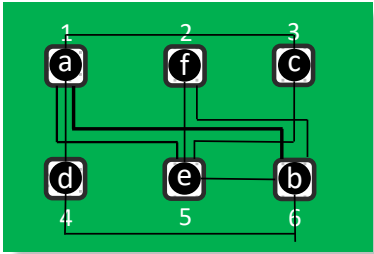
La distance entre les emplacements est la distance de Manhattan :
 $d(1,2)=1$, $d(1,6)=3$, ...

Certains composants doivent être connectés entre-eux par une ou plusieurs connexions. Le nombre de connexions est donné par le tableau suivant

	a	b	c	d	e	f
a	0	12	4	3	7	0
b	12	0	0	6	1	2
c	4	0	0	0	2	1
d	3	6	0	0	0	0
e	7	1	2	0	0	1
f	0	2	1	0	1	0

Tableau symétrique de diagonale nulle

Exemple de positionnement :



$$L = 12 \times 3 + 4 \times 2 + 3 \times 1 + 7 \times 2 + 6 \times 2 + 1 \times 1 + 2 \times 2 + 2 \times 2 + 1 \times 1 + 1 \times 1 = 80$$

Comment placer les composants de manière à minimiser la longueur totale de toutes les connexions ?

```
* (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
```

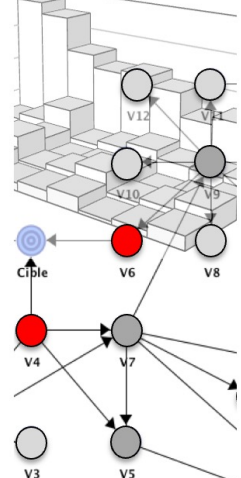
$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

JOB SHOP SCHEDULING PROBLEM

Ordonnancement de tâches dans un atelier de production

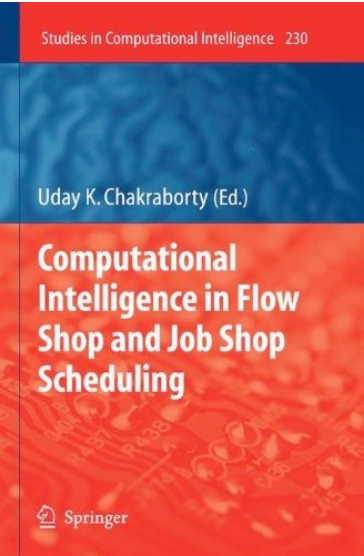
Soient n tâches et m machines.

Trouver un ordonnancement des tâches sur les machines qui minimise le temps total de production

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$


```
*(Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - \dots} \right)$$



ASSIGNMENT PROBLEM

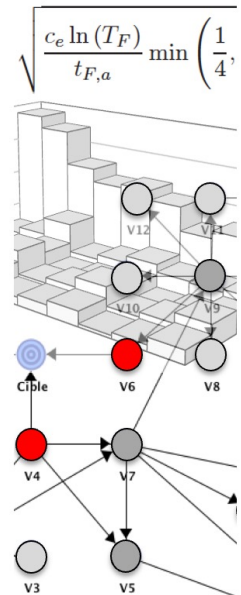


Dans un atelier, en début de journée, m tâches sont prévues (chaque tâche dure la journée entière et est réalisée par un seul ouvrier); les ouvriers présents (car certains peuvent être absents) sont au nombre de n .

On note A_{ij} une valeur entière qui vaut 1 si l'employé j est qualifié pour faire la tâche i , et 0 sinon.

Les tâches non exécutées seront reportées au jour suivant et les employés non affectés (par défaut de qualification) seront affectés à des tâches non rentables (rangement, ...).

Comment exécuter un maximum de tâches pendant la journée ?



```

* (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
  
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

ASSIGNMENT PROBLEM



Modélisation par un programme linéaire en variables binaires :

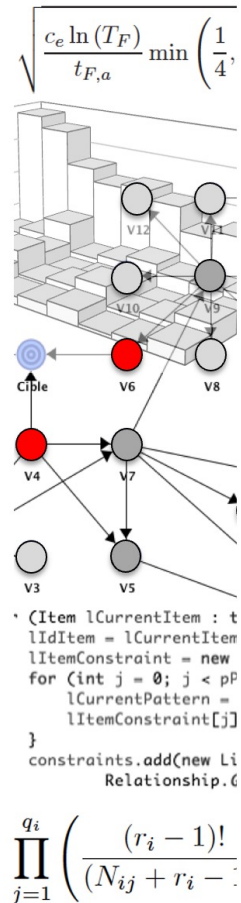
$x_{ij} = 1$ si l'employé j exécute la tâche i et 0 sinon

Un employé j ne peut pas exécuter plus d'une tâche : $\sum_{i=1}^m x_{ij} \leq 1, \forall j$

Une tâche i n'est exécutée que par au plus un ouvrier : $\sum_{j=1}^n x_{ij} \leq 1, \forall i$

Un employé ne peut être affecté qu'à une tâche qu'il sait faire : $x_{ij} \leq A_{ij}, \forall i, j$

Maximiser le nombre de tâches : $\max \sum_{i=1}^m \sum_{j=1}^n x_{ij}$



CHALLENGES DE LA ROADEF

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

```
• (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.p(
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

ROADEF

<http://www.roadef.org/challenge>

Challenge 2022	2020	2018	2016	2014	2012	2010	2009	2007	2005	2003	2001	1999
----------------	------	------	------	------	------	------	------	------	------	------	------	------

[Home](#)
[Schedule](#)
[Subject](#)
[Instances and Checker](#)
[Rules](#)
[Awards](#)
[Organizing Committee](#)
[Registered Teams](#)
[Sprint Results](#)
[Qualification Results](#)

Thanks to the success of the previous challenges, the French Operational Research (OR) and Decision Support Society ([ROADEF](#)) organizes exceptionally jointly with the European Operational Research Society ([EURO](#)) the ROADEF/EURO challenge 2022 dedicated to a 3D truck loading optimization problem in collaboration with [Renault Group](#).

This challenge is **open** to everyone, and particularly to young researchers, excluding people professionally involved with the industrial partners.

The goal of this challenge has multiple aspects.

First, it allows some of our industrial partners to follow recent developments in the fields of Operations Research and Decision Analysis.

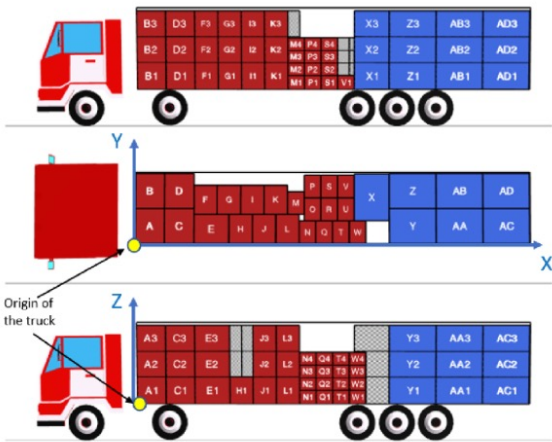
Second, through the **junior category** young researchers have the opportunity to face up to a complex industrial optimization challenge will help them an opportunity to explore the requirements and difficulties encountered in industrial applications. This challenge will help to establish a permanent partnership between manufacturers and young scientists on industrial-size require both high scientific qualification and the real-life practices in companies making use of decision analysis.

Third, through the **senior category**, this challenge allows qualified researchers to demonstrate their knowledge and expertise on the practical problems. This also gives them the opportunity to establish partnerships with industrial partners.

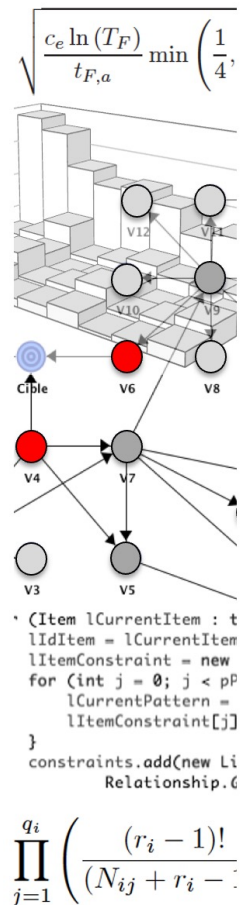
Last, a **scientific prize** dedicated to qualitative submissions is proposed.

ROADEF

THE ASSOCIATION OF EUROPEAN OPERATIONAL RESEARCH SOCIETIES



OPTIMISATION COMBINATOIRE + MACHINE LEARNING



RECENT ADVANCES IN INTEGRATING

MACHINE LEARNING & COMBINATORIAL OPTIMIZATION

Tutorial MH7 at AAAI-21
8:30 am – 11:45 am PST
February 3, 2021

BRIEF OVERVIEW

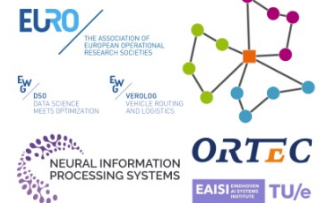
This tutorial will provide an overview of the recent impact machine learning is having on combinatorial optimization, particularly under the Mixed Integer Programming (MIP) framework. Topics covered will include ML and reinforcement learning for predicting feasible solutions, improving exact solvers with ML, a software framework for learning in exact MIP solvers, and the emerging paradigm of decision-focused learning.

<https://sites.google.com/view/ml-co-aaai-21/?pli=1>

Introduction

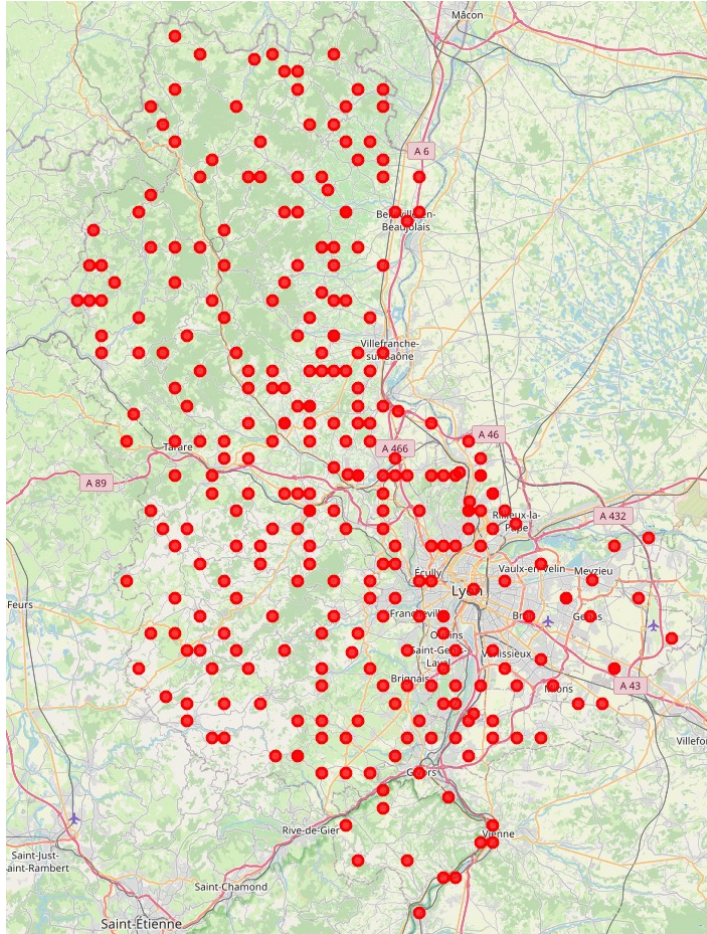
The *EURO Meets NeurIPS 2022 Vehicle Routing Competition* brings together researchers from operations research (OR) and machine learning (ML) to address the vehicle routing problem with time windows (VRPTW) as well as a dynamic VRPTW.

EURO Meets NeurIPS 2022
Vehicle Routing Competition



<https://euro-neurips-vrp-2022.challenges.ortec.com/>

FACILITY LOCATION PROBLEM



Etant donnés l'ensemble des communes du Rhône et un nombre n de « facilities » à positionner, déterminer les n communes où positionner ces « facilities » et affecter chacune des autres communes à une « facility » afin de minimiser la somme totale des distances des communes à la « facility » qui lui attribuée.

1. Modéliser le problème sous forme d'un PLNE
2. Importer les jeux de données depuis Moodle
3. Utiliser un solver pour résoudre le problème

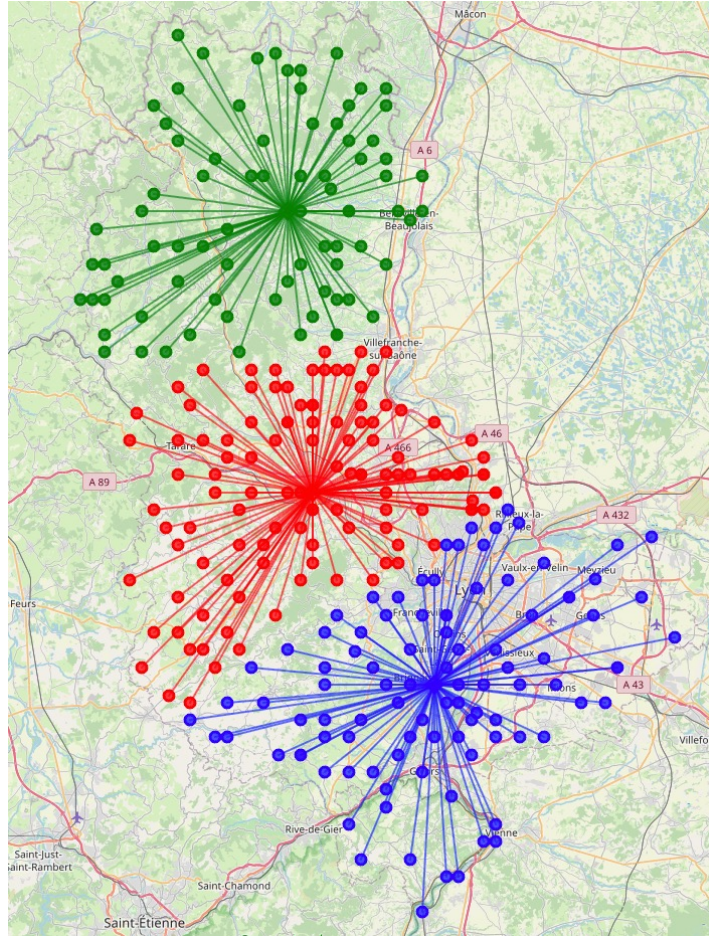
$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min \left(\frac{1}{4} \right)$$

```

* (Item lCurrentItem : t
  lIdItem = lCurrentItem
  lItemConstraint = new
  for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
  }
  constraints.add(new Li
    Relationship.C
  )
  
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)} \right)$$

FACILITY LOCATION PROBLEM

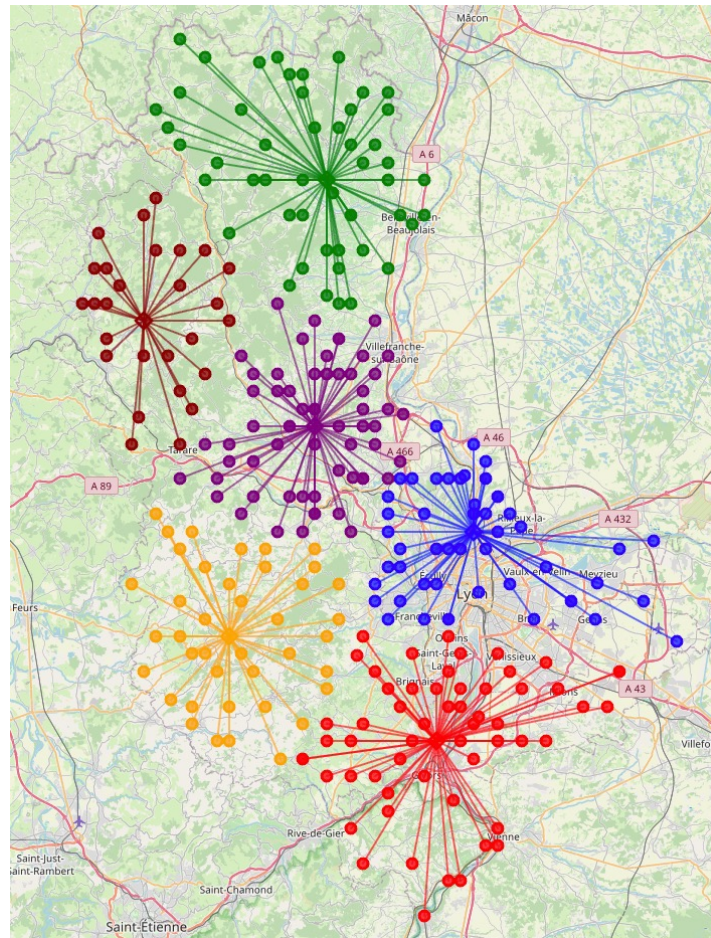


Résultat pour pour le fichier « RhoneCities_500.flpb »

```
NAME: RhoneCities_500.flpb
COMMENT: Cities in the Rhone department with a cost of 500
PROBLEM: Facility Location Problem
NB_FACILITIES: 293
NB_CUSTOMERS: 293
COORDINATES: True
```

```
FACILITIES [capacity cost x y]
1000000 500 45.7167 4.8
1000000 500 45.5833 4.63333
1000000 500 45.75 4.7
1000000 500 46.1167 4.38333
1000000 500 46.0667 4.33333
1000000 500 45.85 4.86667
1000000 500 46.1167 4.73333
1000000 500 45.9333 4.45
1000000 500 45.8333 4.88333
1000000 500 45.8167 4.45
```

FACILITY LOCATION PROBLEM



Résultat pour pour le fichier « RhoneCities_200.flpb »

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

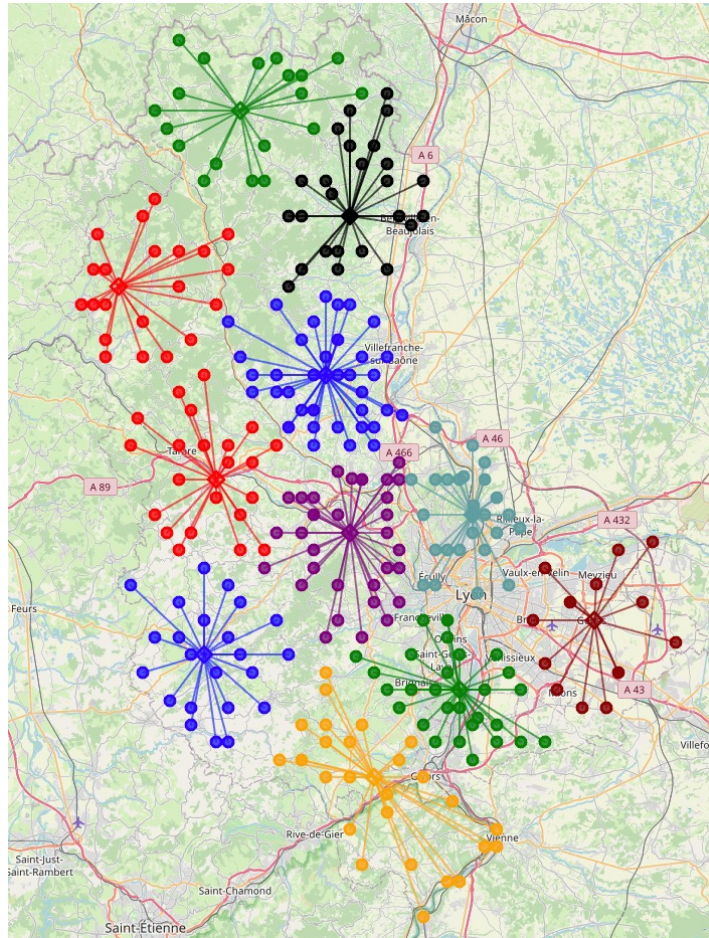
Diagram illustrating a facility location problem with nodes V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, and V11. A target node 'Cible' is shown. The diagram includes a 3D bar chart and a network graph.

```

    (Item lCurrentItem : t
    lIdItem = lCurrentItem
    lItemConstraint = new
    for (int j = 0; j < pP
    lCurrentPattern =
    lItemConstraint[j]
    }
    constraints.add(new Li
    Relationship.C
  
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$

FACILITY LOCATION PROBLEM



Résultat pour pour le fichier « RhoneCities_100.flpb »

$$\sqrt{\frac{c_e \ln(T_F)}{t_{F,a}}} \min\left(\frac{1}{4}, \dots\right)$$

Diagram illustrating a facility location problem with nodes V1 through V12 and a target node Cible. The diagram shows a network of nodes and edges, with a specific path highlighted from V4 to V7 to V5. A 3D bar chart is also shown, representing the cost or distance for different facility locations.

```

    * (Item lCurrentItem : t
      lIdItem = lCurrentItem
      lItemConstraint = new
      for (int j = 0; j < pP
        lCurrentPattern =
        lItemConstraint[j]
      }
      constraints.add(new Li
        Relationship.p.
  
```

$$\prod_{j=1}^{q_i} \left(\frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \right)$$